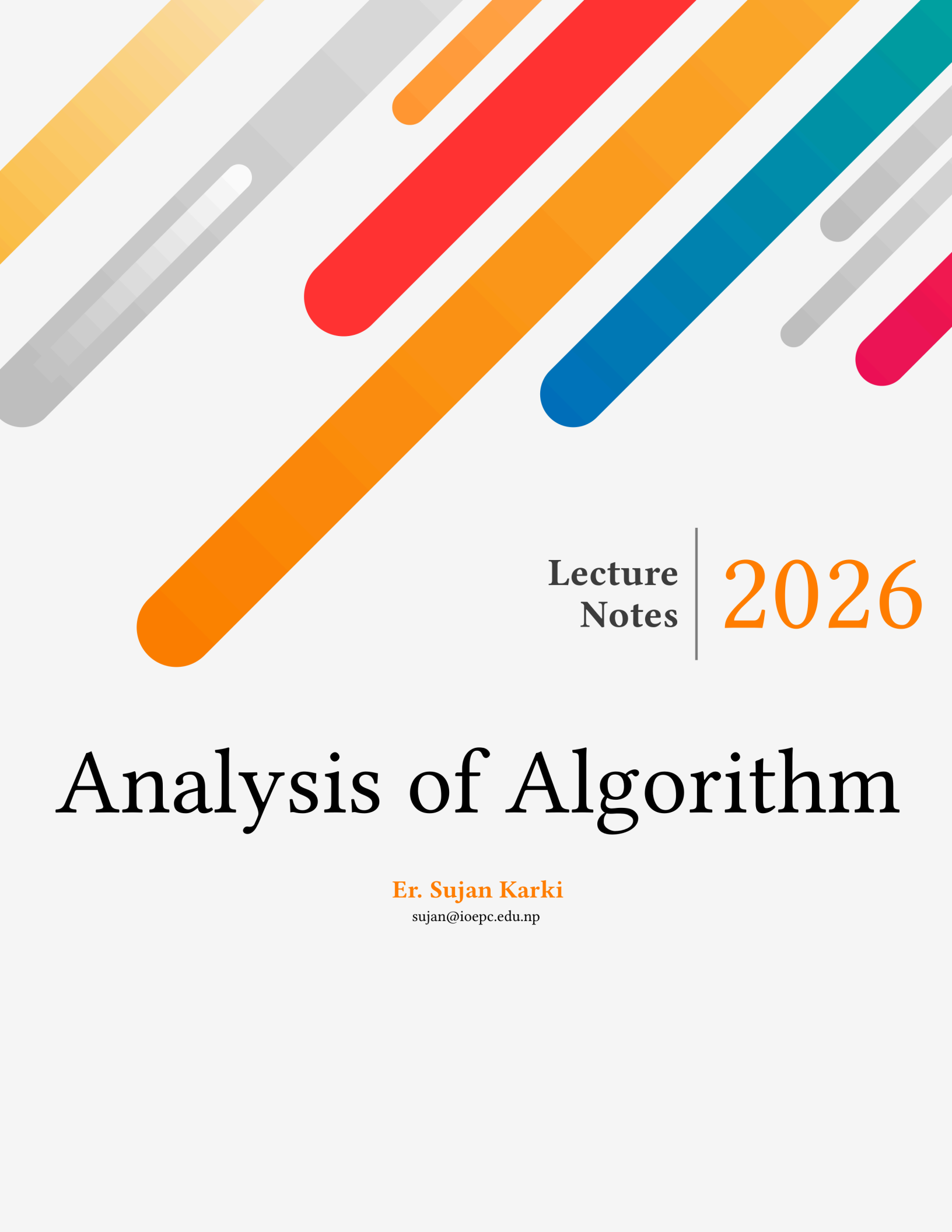




Lecture
Notes

2026

Analysis of Algorithm

Er. Sujan Karki

sujan@ioepc.edu.np

Chapter 4

Greedy Algorithms

Definition 4.0.1

An **optimization problem** is a problem where:

- You are given a set of possible solutions.
- Each solution has a cost or value.
- The goal is to *maximize* or *minimize* that value.

Types of Optimization Problems

1. **Minimization Problem:** Minimize cost, time, distance, weight, etc.

Examples: Minimum spanning tree, Shortest path, Minimum cost job scheduling.

2. **Maximization Problem:** Maximize profit, value, efficiency, etc.

Examples: Knapsack problem, Maximum activity selection, Maximum profit scheduling.

An **optimal solution** is a feasible solution that gives the best possible value (minimum or maximum).

Example 4.0.1

If we want to minimize travel distance:

- Route A = 20 km
- Route B = 15 km
- Route C = 12 km $\Rightarrow$ **Optimal Solution**

Important Terms

- **Feasible Solution:** Any solution that satisfies the constraints.
- **Optimal Solution:** The best among all feasible solutions.
- **Global Optimum:** The best overall solution.
- **Local Optimum:** The best in a small region (may not be global).

4.0.1 Introduction to Greedy Algorithms

Definition 4.0.2

A **greedy algorithm** is an algorithmic technique used to solve optimization problems by building the solution step by step, making decisions based only on the information available at the current moment.

It does not evaluate how the current decision will affect future decisions or the final outcome. Decisions are completely *locally optimal*.

The method constructs the solution simply by looking at the current benefit, without exploring future possibilities — hence the name *greedy*.

Note:-

The choices made under a greedy procedure are **irrevocable**: once we have selected the local best solution, it cannot be backtracked.

Every choice made at each step of the greedy method must be:

- **Feasible**: The choice should satisfy the problem's constraints.
- **Locally Optimal**: The best solution from all feasible solutions at the current stage should be selected.
- **Irrevocable**: Once the choice is made, it cannot be altered. If a feasible solution is selected or rejected in step i , it cannot be reversed in subsequent stages.

4.0.2 Elements (Pillars) of Greedy Strategy

1. **Candidate Set**: A pool of choices from which a selection is made and the solution is constructed. *Examples*: a set of nodes, vertices, items, denominations, etc.
2. **Selection Function**: Chooses the best local optimal solution based on some heuristic (criterion), depending on whether the problem is maximization or minimization.
3. **Feasibility Function**: The candidate selected by the selection function is tested against the feasibility function. It tells us whether to add or not add a particular candidate to the solution set by checking whether it violates the constraints.
4. **Objective Function**: Assigns a benefit or cost to a partial solution. It decides whether a problem is solved by maximizing cost/profit or by minimizing it.
Example: Knapsack is a maximization problem; “make change” is a minimization problem.
5. **Solution Function**: Indicates whether a complete solution has been found. If a complete solution is built, the algorithm stops; otherwise, it continues selecting candidates until a solution is found or the candidate set is exhausted.

Note:-

The greedy technique does not always guarantee the best solution, but it does provide a good estimate in most circumstances. For some problems (such as *Prim's* and *Kruskal's* algorithms), greedy **always** produces the best answer. When greedy gives the best solution, it is the finest technique among all others.

4.0.3 Characteristics of Greedy Algorithms

Problems that can be solved using the greedy method generally possess the following two important properties:

1. **Greedy Choice Property**: The global optimal solution is found by selecting locally optimal choices — the ones that appear to be the best at the time. If the choice is feasible, include it in the solution set and reduce the problem by that amount.
2. **Optimal Substructure**: A given problem exhibits optimal substructure if the optimal solution to the problem contains the optimal solutions to its subproblems too.

This property ensures that making a locally optimal choice at each step (the greedy choice) still leads to a globally optimal outcome, because the remaining problem after that choice also has an optimal solution that fits together with the choice made.

4.0.4 Popular Problems Solved by Greedy Algorithm

Greedy algorithms are used to find an optimal or near-optimal solution to many real-life problems. A few of them are listed below :

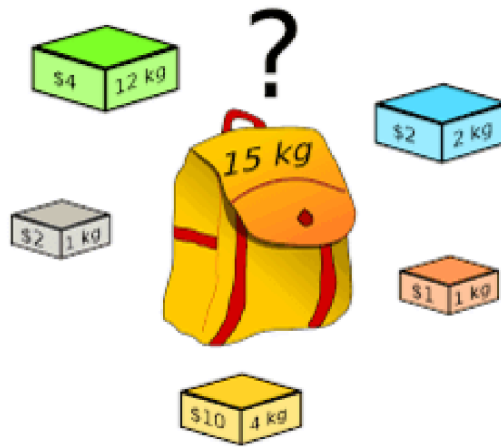
1. Binary Knapsack Problem
2. Fractional Knapsack Problem
3. Job Scheduling Problem
4. Activity Selection Problem
5. Huffman Coding
6. Optimal Storage on Tapes
7. Optimal Merge Pattern
8. Prim's Algorithm
9. Kruskal's Algorithm
10. Dijkstra's Algorithm

4.1 Fractional Knapsack Problem

Definition 4.1.1

Given a set of items having some weight and value/profit associated with them, the knapsack problem is to find a set of items such that the total weight is less than or equal to a given limit (size of knapsack) and the total value/profit earned is as large as possible.

4.1.1 Problem Formulation



Let $X = \langle x_1, x_2, x_3, \dots, x_n \rangle$ be the set of n items. $W = \langle w_1, w_2, w_3, \dots, w_n \rangle$ and $V = \langle v_1, v_2, v_3, \dots, v_n \rangle$ are the sets of weights and values associated with each item, respectively. Knapsack capacity is M . Select items one by one from X and fill the knapsack such that it maximizes the value.

Two Variants of the Knapsack Problem

- **0/1 Knapsack (Binary Knapsack):** Does NOT allow breaking of items. Either add the entire item to the knapsack or reject it.
- **Fractional Knapsack:** Allows the breaking of items. Profit will also be considered fractionally.

Greedy Strategy

The greedy approach selects items in decreasing order of their **profit-to-weight ratio**:

$$p_i = \frac{v_i}{w_i}$$

4.1.2 Algorithm

Algorithm 4.1.1

GREEDY_FRACTIONAL_KNAPSACK(X, V, W, M)

Input:

X: An array of n items

V: An array of profits associated with each item

W: An array of weights associated with each item

M: Capacity of knapsack **Output:** SW (weight of selected items), SP (profit of selected items) **Precondition:** Items are presorted in decreasing order of $p_i = v_i/w_i$

1. $S \leftarrow \emptyset$ (set of selected items, initially empty)
2. $SW \leftarrow 0$ (weight of selected items)
3. $SP \leftarrow 0$ (profit of selected items)
4. $i \leftarrow 1$
5. While $i \leq n$:
 - (a) If $(SW + W[i]) \leq M$:
 - i. $S \leftarrow S \cup X[i]$
 - ii. $SW \leftarrow SW + W[i]$
 - iii. $SP \leftarrow SP + V[i]$
 - (b) Else:
 - i. $frac \leftarrow (M - SW)/W[i]$
 - ii. $S \leftarrow S \cup X[i] \times frac$ (add fraction of item)
 - iii. $SP \leftarrow SP + V[i] \times frac$ (add fraction of profit)
 - iv. $SW \leftarrow SW + W[i] \times frac$ (add fraction of weight)
 - (c) $i \leftarrow i + 1$
6. Return (S, SW, SP)

4.1.3 Time Complexity

Step 1: Sorting the Items

Items are presorted in decreasing order of $p_i = v_i/w_i$. Using merge sort or heap sort, n items can be sorted in $O(n \log_2 n)$ time.

Step 2: While Loop Execution

The loop runs at most n times. Each iteration performs:

- Comparison $\rightarrow O(1)$
- Addition / multiplication $\rightarrow O(1)$
- Assignment $\rightarrow O(1)$

Hence, the total loop time is $O(n)$.

Step 3: Return Statement

The cost of return is constant time $\rightarrow O(1)$.

$$T(n) = O(n \log n)$$

4.1.4 Space Complexity

The algorithm uses only a constant number of extra variables:

- S (set of selected items – can be represented implicitly, or we only track total profit/weight),
- SW (total weight),
- SP (total profit),
- i (loop counter),
- $frac$ (fraction of an item).

No recursion, no additional arrays that depend on n are allocated. The input arrays X , V , W are not counted as auxiliary space.

Therefore,

$$S(n) = O(1).$$

(The algorithm uses constant auxiliary space regardless of input size.)

Example 4.1.1

Consider the fractional knapsack problem with $n = 3$, $M = 20$, $V = (24, 25, 15)$ and $W = (18, 15, 20)$. Find the feasible solution.

Step 1: Initial Table

Items (X)	W	V
I_1	18	24
I_2	15	25
I_3	20	15

Step 2: Calculate V/W Ratio

Items (X)	W	V	V/W
I_1	18	24	$24/18 = 1.33333$
I_2	15	25	$25/15 = 1.66667$
I_3	20	15	$15/20 = 0.75$

Step 3: Arrange in Decreasing Order of p_i

Items (X)	W	V	$p_i = V/W$
I_2	15	25	1.66667
I_1	18	24	1.33333
I_3	20	15	0.75

Note:-

We shall select one by one item from Table. If the inclusion of an item does not cross the knapsack capacity, then add it. Otherwise, break the current item and select only the portion of item equivalent to remaining knapsack capacity. Select the profit accordingly. We should stop when knapsack is full or all items are scanned.

Initialize: $SW = 0$, $SP = 0$, $S = \{\}$, $M = 20$.

Iteration 1: $i = 1$

$(SW + W[1]) \leq M \Rightarrow (0 + 15) \leq 20$ (TRUE)

- $S \leftarrow S \cup X[i]$ $S \leftarrow \{\} \cup I_2$
- $SW \leftarrow SW + W[i]$ $SW \leftarrow 0 + W[1] = 0 + 15 = 15$
- $SP \leftarrow SP + V[i]$ $SP \leftarrow 0 + V[1] = 0 + 25 = 25$

Iteration 2: $i = 2$

$(SW + W[2]) \leq M \Rightarrow (15 + 18) \leq 20$ (FALSE)

- $frac \leftarrow (20 - 15)/18 = 5/18 = 0.277778$
- $S \leftarrow \{I_2, I_1 \times 0.277778\}$
- $SP \leftarrow 25 + 24 \times 0.277778 = 31.66667$
- $SW \leftarrow 15 + 18 \times 0.277778 = 20$

The knapsack is now full.

Final Solution: $S = \{I_2, I_1 \times 0.277778\}$, Profit = 31.67 units, Weight = 20 units

Question 4.1

Find the optimal solution for the fractional knapsack problem where knapsack capacity = 28, $P = \{9, 5, 2, 7, 6, 16, 3\}$ and $W = \{2, 5, 6, 11, 1, 9, 1\}$.

Answer: $S = \{I_5, I_1, I_7, I_6, I_2, 0.91 \times I_4\}$, $SW = 28$, $SP = 45.37$.

4.2 Job Sequencing with Deadlines

Definition 4.2.1

Job scheduling is the problem of scheduling jobs out of a set of N jobs on a single processor which maximizes profit as much as possible.

Given N jobs, each taking unit time for execution on a single processor. Each job has a profit and a deadline associated with it. Profit is earned only if the job is completed on or before its deadline. Each job has deadline $d_i \geq 1$ and profit $p_i \geq 0$. At any time, only one job can be active on the processor.

A job is **feasible** only if it can be finished on or before its deadline. A feasible solution is a subset of N jobs such that each job can be completed on or before its deadline. An **optimal solution** is one with maximum profit.

In simple terms,

We can think of the schedule S consisting of a sequence of job slots $1, 2, 3, \dots, N$, where $S(t)$ indicates the job scheduled in slot t . Slot t has a span of $(t - 1)$ to t .

- $S(t) = 0$ implies no job is scheduled in slot t .
- Schedule S is feasible if: when $S(t) = i$, then $t \leq d_i$ (i.e., Scheduled job must be completed by its deadline).
- Each job can be scheduled at most once.

4.2.1 Algorithm

Algorithm 4.2.1

JOB_SCHEDULING(J, D, P)

Input:

J : Array of N jobs

D : Array of deadlines for each job

P : Array of profits associated with each job

Output: Set of scheduled jobs which gives maximum profit

Sort all jobs in J in decreasing order of profit

1. $S \leftarrow \emptyset$ (set of scheduled jobs)
2. $SP \leftarrow 0$ (sum of profits earned)
3. For $i \leftarrow 1$ to N :
 - (a) If Job $J[i]$ is feasible:
 - i. Schedule the job in the latest possible free slot meeting its deadline
 - ii. $S \leftarrow S \cup J[i]$
 - iii. $SP \leftarrow SP + P[i]$
4. Return (S, SP)

4.2.2 Time Complexity

Step 1: Sorting Jobs

Sorting N jobs by profit using merge sort or quicksort takes $O(n \log_2 n)$.

Step 2: Scheduling Jobs

For each job i :

- **Check feasibility:** Scan all slots from deadline $D[i]$ down to 1 to find the latest free slot. In the worst case, this checks all N slots per job.
- **Schedule job:** $O(1)$ to mark the slot as occupied.

So, total time for one job is $O(n)$. For all n jobs, total time is $O(n^2)$.

Total Time Complexity

$$T(n) = O(n \log_2 n) + O(n^2) = O(n^2)$$

Total Time Complexity = $O(n^2)$

(With Disjoint Set / Union-Find: nearly $O(N)$)

4.2.3 Space Complexity

The input arrays J , D , and P are not counted in auxiliary space because they are provided as input to the algorithm. The extra memory used by the algorithm is:

- Set S for storing scheduled jobs $\rightarrow O(N)$. Because in the worst case, all jobs could be scheduled if their deadlines allow it, so S can grow up to size N .
- Slot tracking array (to check free/occupied slots) $\rightarrow O(N)$. Because the maximum deadline can be at most N (since we have N jobs), we can use an array of size N to track which slots are occupied. Each index in this array corresponds to a time slot, and we mark it as occupied when a job is scheduled there.

Since both the set S and the slot tracking array can grow up to $O(N)$, the total space complexity is:

$$O(N) + O(N) = O(N)$$

Therefore,

Space Complexity = $O(N)$

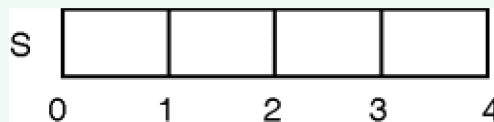
Example 4.2.1

Solve the job scheduling problem with deadlines using the greedy method.

$N = 4$, Profits $(P_1, P_2, P_3, P_4) = (100, 10, 15, 27)$, Deadlines $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$.

Step 1: Sort Jobs by Decreasing Profit

$$P = (100, 27, 15, 10), \quad J = (J_1, J_4, J_3, J_2), \quad D = (2, 1, 2, 1)$$



Initialize: $S = \{\}$, $SP = 0$.

Iteration 1: Job J_1 (Deadline 2)

Slot 2 ($t = 1$ to $t = 2$) is free $\Rightarrow$ schedule J_1 in slot 2.

- $S = \{J_1\}$
- $SP = 0 + 100 = 100$

Iteration 2: Job J_4 (Deadline 1)

Slot 1 ($t = 0$ to $t = 1$) is free $\Rightarrow$ schedule J_4 in slot 1.

- $S = \{J_1, J_4\}$
- $SP = 100 + 27 = 127$

Iteration 3: Job J_3 (Deadline 2)

The first two slots are already occupied. J_3 cannot be finished before its deadline 2. **Discarded.**

Iteration 4: Job J_2 (Deadline 1)

Slot 1 is already occupied, and J_2 cannot be finished before deadline 1. **Discarded.**

Final Schedule: $S = \{J_4, J_1\}$ (in order of execution) Total Profit: $100 + 27 = 127$ units

Example 4.2.2

Solve the job scheduling problem with deadlines using the greedy method.

$N = 7$, Jobs with (Deadline, Profit):

$(J_1 : 4, 70), (J_2 : 2, 60), (J_3 : 1, 30), (J_4 : 4, 50), (J_5 : 6, 10), (J_6 : 3, 40), (J_7 : 4, 20)$

Step 1: Sort Jobs by Decreasing Profit

Job	Profit	Deadline
J_1	70	4
J_2	60	2
J_4	50	4
J_6	40	3
J_3	30	1
J_7	20	4
J_5	10	6

Initialize: $S = \{\}$, $SP = 0$, time slots 1, 2, 3, 4, 5, 6 (max deadline = 6).

Iteration 1: Job J_1 (Deadline 4, Profit 70)

Latest free slot ≤ 4 is slot 4. Schedule J_1 in slot 4.

- $S = \{J_1\}$, $SP = 70$
- Occupied slots: $\{4\}$

Iteration 2: Job J_2 (Deadline 2, Profit 60)

Latest free slot ≤ 2 is slot 2. Schedule J_2 in slot 2.

- $S = \{J_1, J_2\}$, $SP = 70 + 60 = 130$
- Occupied slots: $\{2, 4\}$

Iteration 3: Job J_4 (Deadline 4, Profit 50)

Latest free slot ≤ 4 : slot 4 is taken, try slot 3 (free). Schedule J_4 in slot 3.

- $S = \{J_1, J_2, J_4\}$, $SP = 130 + 50 = 180$
- Occupied slots: $\{2, 3, 4\}$

Iteration 4: Job J_6 (Deadline 3, Profit 40)

Latest free slot ≤ 3 : slot 3 taken, slot 2 taken, slot 1 is free. Schedule J_6 in slot 1.

- $S = \{J_1, J_2, J_4, J_6\}$, $SP = 180 + 40 = 220$
- Occupied slots: $\{1, 2, 3, 4\}$

Iteration 5: Job J_3 (Deadline 1, Profit 30)

Latest free slot ≤ 1 : slot 1 is already occupied $\rightarrow$ no free slot. **Discarded.**

Iteration 6: Job J_7 (Deadline 4, Profit 20)

All slots 1–4 are occupied $\rightarrow$ no free slot ≤ 4 . **Discarded.**

Iteration 7: Job J_5 (Deadline 6, Profit 10)

Latest free slot ≤ 6 : slots 5 and 6 are free; choose slot 6 (latest). Schedule J_5 in slot 6.

- $S = \{J_1, J_2, J_4, J_6, J_5\}$, $SP = 220 + 10 = 230$
- Occupied slots: $\{1, 2, 3, 4, 6\}$

Final Schedule (by time slot): J_6, J_2, J_4, J_1, J_5 Total Profit: $70 + 60 + 50 + 40 + 10 = 230$ units

Question 4.2

Solve: $n = 7$, Profits $(p_1, \dots, p_7) = (3, 5, 20, 18, 1, 6, 30)$, Deadlines $(d_1, \dots, d_7) = (1, 3, 4, 3, 2, 1, 2)$.

Answer: $\{J_7, J_3, J_4, J_6\}$, Profit = 74 units.

4.3 Analysis of Minimum Spanning Tree Algorithms

Definition 4.3.1

Given a weighted, connected, undirected graph $G = \langle V, E, W \rangle$, a **spanning tree** is a subgraph that includes all vertices of G with the minimum number of edges (exactly $|V| - 1$) and no cycles. A **minimum spanning tree** is a spanning tree of minimum total edge weight.

Two famous greedy algorithms to find MSTs are:

- **Kruskal's Algorithm** — edge-based, picks minimum weight edges.
- **Prim's Algorithm** — vertex-based, grows the MST from a starting vertex.

4.3.1 Kruskal's Algorithm

Joseph Kruskal suggested a greedy approach to find MSTs from a weighted, undirected graph.

4.3.2 Working Principle

- Sort all edges in non-decreasing order of their weight.
- Select the edge with the minimum weight and test its feasibility.
- If adding the edge to the partial solution does NOT form a cycle, it is feasible and added.
- Otherwise, skip the edge and check the next one.
- Repeat until all edges are scanned (or $|V| - 1$ edges are added).

Algorithm 4.3.1

KRUSKAL_MST(G)

Input: Weighted, undirected, connected graph $G = \langle V, E, W \rangle$

Output: Minimum spanning tree (MST)

1. Let $UE = \{e_1, e_2, \dots, e_m\}$ be the edges sorted in increasing order of weight
2. Initialize $MST \leftarrow \emptyset$
3. While $UE \neq \emptyset$ and $|MST| < |V| - 1$:
 - (a) Select minimum weight edge e_{min} from UE
 - (b) If adding e_{min} to MST does NOT form a cycle:
 - i. $MST \leftarrow MST \cup \{e_{min}\}$
 - (c) Remove e_{min} from UE
4. Return MST

Example 4.3.1 (Kruskal's Algorithm Example)

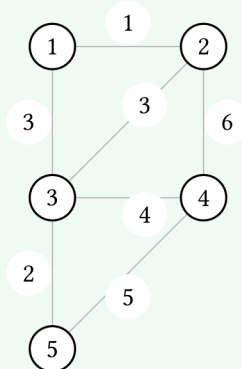
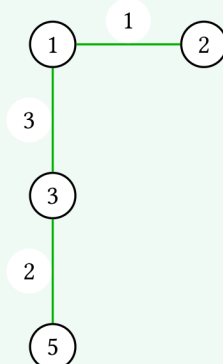
Find the minimum spanning tree of the given graph using Kruskal's Algorithm.

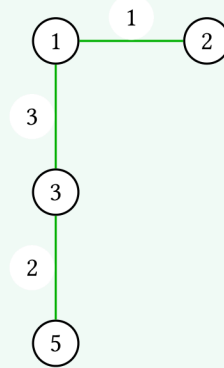
Vertices: $\{1, 2, 3, 4, 5\}$

Edges: $\langle 1, 2 \rangle : 1, \langle 1, 3 \rangle : 3, \langle 2, 3 \rangle : 3, \langle 2, 4 \rangle : 6, \langle 3, 4 \rangle : 4, \langle 3, 5 \rangle : 2, \langle 4, 5 \rangle : 5$.

Sorted Edges (Increasing Order of Weight)

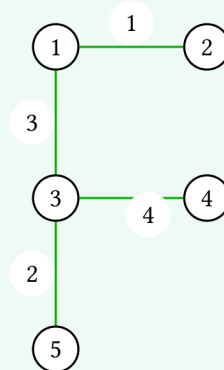
Edge	$\langle 1, 2 \rangle$	$\langle 3, 5 \rangle$	$\langle 1, 3 \rangle$	$\langle 2, 3 \rangle$	$\langle 3, 4 \rangle$	$\langle 4, 5 \rangle$	$\langle 2, 4 \rangle$
Cost	1	2	3	3	4	5	6

Step-by-Step Execution (Only MST is shown)**Step 0: Initial state.** No edges selected yet.*No edges added yet.***Step 1: Process edge $\langle 1, 2 \rangle$ (weight 1).** Vertices 1 and 2 are in different components $\rightarrow$ add to MST.*MST after step 1: edge (1,2) added.***Step 2: Process edge $\langle 3, 5 \rangle$ (weight 2).** Vertices 3 and 5 are in different components $\rightarrow$ add to MST.*MST after step 2: edges (1,2) and (3,5).***Step 3: Process edge $\langle 1, 3 \rangle$ (weight 3).** Vertex 1 (in component $\{1, 2\}$) and vertex 3 (in component $\{3, 5\}$) are different $\rightarrow$ add.*MST after step 3: edges (1,2), (3,5), (1,3).***Step 4: Process edge $\langle 2, 3 \rangle$ (weight 3).** Both endpoints (2 and 3) are already in the same component (connected via 1-2 and 1-3) $\rightarrow$ cycle $\rightarrow$ **reject**. No change to MST.



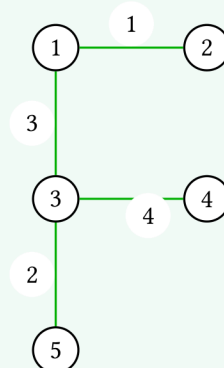
Edge (2,3) rejected (cycle). MST unchanged.

Step 5: Process edge $\langle 3, 4 \rangle$ (weight 4). Vertex 4 is isolated, vertex 3 is in the large component $\rightarrow$ different components $\rightarrow$ add.



MST after step 5: added edge (3,4).

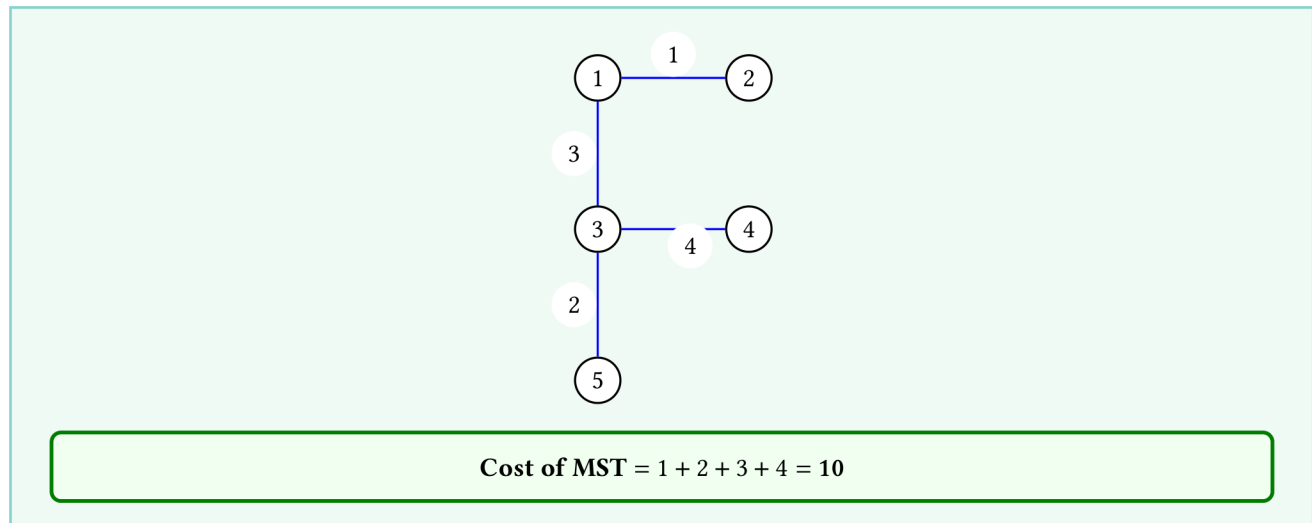
Step 6: Process edge $\langle 4, 5 \rangle$ (weight 5). Now vertices 4 and 5 are already connected (via 4-3-5) $\rightarrow$ cycle $\rightarrow$ **reject**. MST is complete (4 edges). Furthermore, edge $\langle 2, 4 \rangle$ (weight 6) would also create a cycle, but the algorithm stops because $|MST| = |V| - 1 = 4$.



Edge (4,5) rejected (cycle). MST already complete.

Final Minimum Spanning Tree

MST edges: $\langle 1, 2 \rangle$ (1), $\langle 3, 5 \rangle$ (2), $\langle 1, 3 \rangle$ (3), $\langle 3, 4 \rangle$ (4). Total weight = $1 + 2 + 3 + 4 = 10$.



4.3.3 Time Complexity of Kruskal's Algorithm

Let $|V|$ be the number of vertices and $|E|$ the number of edges.

1. **Sorting edges:** Kruskal's algorithm sorts all edges by weight using a comparison-based sorting algorithm. Sorting $|E|$ edges takes $O(|E| \log |E|)$.
2. **Selection & cycle checking (Union-Find):** For each edge, we check whether adding it forms a cycle using a Disjoint Set (Union-Find). The check is nearly $O(1)$ per edge (inverse Ackermann function, effectively constant). Thus for all $|E|$ edges, the total time is $O(|E|)$.
3. **Total time:**

$$T(|E|, |V|) = O(|E| \log |E|) + O(|E|) = O(|E| \log |E|).$$

For a graph, $|E| \leq |V|^2$. Taking logarithms:

$$\log |E| \leq \log(|V|^2) = 2 \log |V|.$$

Ignoring the constant factor, $\log |E| = O(\log |V|)$ (especially for dense graphs). Hence,

$$T(|E|, |V|) = O(|E| \log |V|).$$

4.3.4 Space Complexity of Kruskal's Algorithm

- The algorithm stores the list of edges: $O(|E|)$.
- It uses Union-Find data structures for $|V|$ vertices (parent and rank arrays): $O(|V|)$.
- No other significant memory is required.

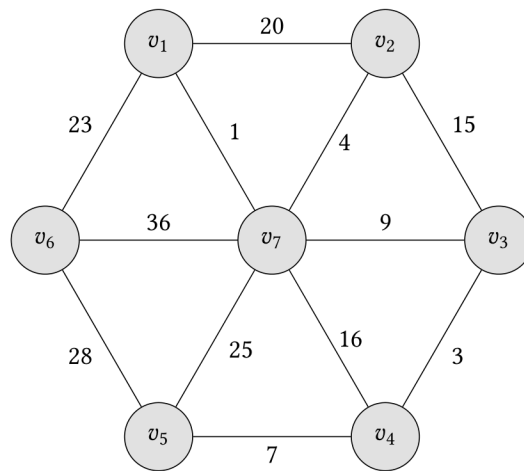
$$S(|V|, |E|) = O(|V| + |E|).$$

Note:-

In practice, $|E|$ is often larger than $|V|$, so space is dominated by $O(|E|)$. The Union-Find operations are extremely efficient, making Kruskal's algorithm a good choice for sparse graphs.

Question 4.3

Find out the minimum cost-spanning tree using the KRUSKAL algorithm.



4.3.5 Prim's Algorithm

Definition 4.3.2

Prim's algorithm is a greedy method to find a minimum spanning tree from a weighted, connected, undirected graph. It starts from an arbitrary vertex and grows the MST by adding the minimum weight edge that connects a vertex in the MST to a vertex outside the MST, until all vertices are included. It is similar in structure to Dijkstra's algorithm, but:

- **Prim's** minimizes the total edge weight of the tree.
- **Dijkstra's** minimizes the distance from a source.

4.3.6 Working Principle

Let the graph have vertices $V = \{V_1, V_2, V_3, \dots, V_n\}$.

Initialization:

- Choose any arbitrary vertex as the starting vertex.
- Let $MST = \{\text{starting vertex}\}$.
- Let $UV = V - \{\text{starting vertex}\}$ (unvisited vertices).

Steps:

1. **Form Candidate Edge Set:** Create a set NE consisting of all edges that connect a vertex in MST to a vertex in UV .
2. **Greedy Selection:** Select the edge $e = (u, v)$ from NE with minimum weight, where $u \in MST$ and $v \in UV$.
3. **Add Edge:** Add edge e to MST . Then add v to MST and remove it from UV .
4. **Repeat:** Repeat steps 1–3 until UV becomes empty.

When all vertices are included, the constructed graph is the **Minimum Spanning Tree**.

Note:-

When to Use Prim vs. Kruskal?

Prim's algorithm is preferred when the graph is *dense* ($|E| \approx |V|^2$).

Kruskal's algorithm performs better on *sparse* graphs ($|E| \approx |V|$).

4.3.7 Algorithm

Assuming the graph has n vertices and m edges.

Algorithm 4.3.2

Prim(G, V, E, w)

Input: Graph G with vertex set V ($|V| = n$), edge set E ($|E| = m$)

Output: MST_Edges — set of edges forming the Minimum Spanning Tree

1. Initialization:

- (a) Choose any arbitrary vertex $s \in V$ $O(1)$
- (b) $MST \leftarrow \{s\}$ $O(1)$
- (c) $UV \leftarrow V - \{s\}$ $O(1)$
- (d) $MST_Edges \leftarrow \emptyset$ $O(1)$
- (e) $total_weight \leftarrow 0$ $O(1)$

2. Main Loop (While $UV \neq \emptyset$): runs $n - 1$ times

- (a) **Form Candidate Edge Set:**
 $NE \leftarrow \{(u, v) \in E \mid u \in MST \text{ and } v \in UV\}$ $O(m)$ per iteration
- (b) **Greedy Selection (minimum-weight edge):**
 $e(u, v) \leftarrow \text{edge in } NE \text{ with minimum } w(u, v)$ $O(m)$ per iteration
- (c) **Add Edge to MST:**
 $MST_Edges \leftarrow MST_Edges \cup \{e(u, v)\}$ $O(1)$
 $total_weight \leftarrow total_weight + w(u, v)$ $O(1)$
- (d) **Move v from UV to MST :**
 $MST \leftarrow MST \cup \{v\}$ $O(1)$
 $UV \leftarrow UV - \{v\}$ $O(1)$

3. Return $MST_Edges, total_weight$ $O(1)$

4.3.8 Time Complexity of Prim's Algorithm

Note:-

The algorithm does not specify how to find the cheapest edge, how to store the graph, or how to track which vertices are visited. Each of these implementation choices changes the cost of every step, and therefore the total time complexity. That is why the same algorithm has multiple complexities — $O(n \cdot m)$, $O(n^2)$, $O((n + m) \log n)$, $O(m + n \log n)$ — depending on the data structures used.

Step	Operation	Cost per iteration	Total
Init	Initialization	$O(1)$	$O(1)$
1	Form candidate edge set NE	$O(m)$	$O(n \cdot m)$
2	Select minimum-weight edge	$O(m)$	$O(n \cdot m)$
3	Add edge to MST	$O(1)$	$O(n)$
4	Move vertex from UV to MST	$O(1)$	$O(n)$

Overall Time Complexity

The dominant cost comes from scanning the edges in Steps 1 and 2 during every iteration of the while loop:

$$T(n, m) = O(n \cdot m).$$

In the worst case (a dense graph where $m \approx n^2$):

$$T(n) = O(n \cdot n^2) = O(n^3).$$

Thus, the naïve implementation of Prim's algorithm runs in $O(n \cdot m)$, or $O(n^3)$ for dense graphs.

Optimized Versions of Prim's Algorithm

The basic version is slow because it rescans all candidate edges in every iteration. Smarter data structures let us avoid this by maintaining the best known edge weight to each unvisited vertex ($\text{key}[v]$) and quickly fetching the minimum.

Version	Data Structure	Find-Min	Key Update	Total Time
1	Adjacency Matrix + Linear Search	$O(n)$	$O(n)$	$O(n^2)$
2	Adjacency List + Binary Heap	$O(\log n)$	$O(\log n)$	$O((n + m) \log n)$
3	Adjacency List + Fibonacci Heap	$O(\log n)$	$O(1)$	$O(m + n \log n)$

4.3.9 Space Complexity of Prim's Algorithm

Space complexity measures the **extra memory** the algorithm uses, beyond the input graph itself.

Let: n = number of vertices and m = number of edges

Step-by-Step Space Usage (Naive Version)

Variable / Structure	Purpose	Space
MST	Set of visited vertices	$O(n)$
UV	Set of unvisited vertices	$O(n)$
MST_Edges	Edges in the MST ($n - 1$ edges)	$O(n)$
NE	Candidate edge set (worst case all edges)	$O(m)$
$total_weight$	Single number	$O(1)$

Total auxiliary space = $O(n + m)$

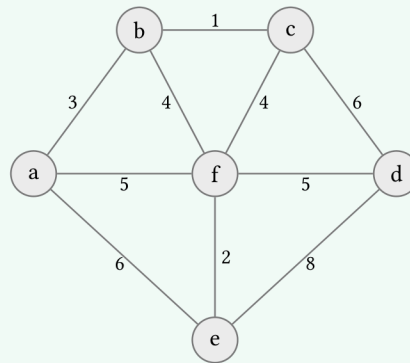
Space Complexity by Implementation

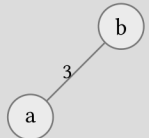
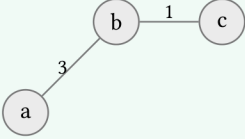
#	Implementation	Graph Storage	Auxiliary Data	Total Space
1	Naive (edge list)	$O(m)$	$O(n + m)$	$O(n + m)$
2	Adjacency Matrix + Linear Search	$O(n^2)$	$O(n)$	$O(n^2)$
3	Adjacency List + Binary Heap	$O(n + m)$	$O(n)$	$O(n + m)$
4	Adjacency List + Fibonacci Heap	$O(n + m)$	$O(n)$	$O(n + m)$

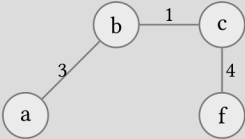
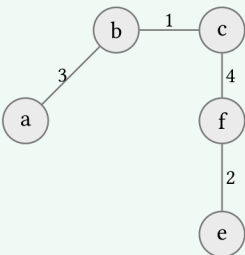
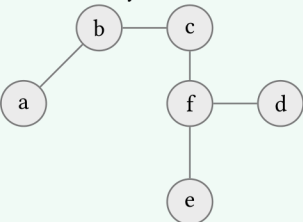
So just like time complexity, **space complexity also depends on the implementation**, not on the algorithm's logic.

Example 4.3.2

Find the cost of minimum spanning tree of the given graph by using Prim's algorithm.

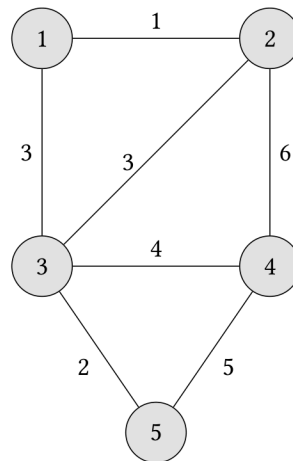


Partial solution	Set of neighbour edges N_E	Cost of neighbour edges	Updated U_V
Step 1 : $U_V = \{a, b, c, d, e, f\}$. Let us start with arbitrary vertex a . Initial partial solution 	$\langle a, b \rangle$ $\langle a, f \rangle$ $\langle a, e \rangle$	3 5 6	$\{b, c, d, e, f\}$
Step 2 : Edge $\langle a, b \rangle$ has minimum cost, so add it. 	$\langle a, f \rangle$ $\langle a, e \rangle$ $\langle b, c \rangle$ $\langle b, f \rangle$	5 6 1 4	$\{c, d, e, f\}$
Step 3 : Edge $\langle b, c \rangle$ has a minimum cost, so add it. 	$\langle a, f \rangle$ $\langle a, e \rangle$ $\langle b, f \rangle$ $\langle c, f \rangle$ $\langle c, d \rangle$	5 6 4 4 6	$\{d, e, f\}$

Partial solution	Set of neighbour edges N_E	Cost of neighbour edges	Updated U_V
Step 4 : Edge $\langle b, f \rangle$ and $\langle c, f \rangle$ have same minimum cost, so we can add any of it. Let us add $\langle c, f \rangle$ 	$\langle a, f \rangle$ $\langle a, e \rangle$ $\langle b, f \rangle$ $\langle c, d \rangle$ $\langle f, d \rangle$ $\langle f, e \rangle$	5 6 4 6 5 2	$\{d, e\}$
Step 5 : Edge $\langle f, e \rangle$ has a minimum cost, so add it. 	$\langle a, f \rangle$ $\langle a, e \rangle$ $\langle b, f \rangle$ $\langle c, d \rangle$ $\langle f, d \rangle$	5 6 4 6 5	$\{d\}$
Step 6 : Edge $\langle b, f \rangle$ has minimum cost, but its inclusion in above partial solution creates cycle, so skip edge $\langle b, f \rangle$ and check for next minimum cost edge i.e. $\langle a, f \rangle$ Inclusion of $\langle a, f \rangle$ also creates cycle so skip it and check for next minimum cost edge, i.e. $\langle f, d \rangle$ The inclusion of $\langle f, d \rangle$ does not create a cycle, so it is a feasible edge, add it. 			U_V is empty so the tree generated in step 6 is the minimum spanning tree of given graph. Let $w(u, v)$ represent the weight of edge (u, v). Cost of solution: $w(a, b) + w(b, c) + w(c, f) + w(f, d) + w(f, e) = 3 + 1 + 4 + 5 + 2 = 15$

Question 4.4

Find the cost of Minimal Spanning Tree of the given graph by using Prim's Algorithm.



4.4 Analysis of Single Source Shortest Path Algorithm

Definition 4.4.1

Given a weighted, directed (or undirected) graph $G = \langle V, E, W \rangle$ and a source vertex $s \in V$, find the shortest path from s to every other vertex $v \in V$.

4.4.1 Dijkstra's Algorithm

Definition 4.4.2

Given a weighted, directed (or undirected) graph $G = \langle V, E, W \rangle$ and a source vertex $s \in V$, find the shortest path from s to every other vertex $v \in V$.

Note:-

A breakthrough 2025 algorithm by Ran Duan (Tsinghua University) has surpassed Dijkstra's algorithm for the Single-Source Shortest Path (SSSP) problem, breaking a 40-year-old "sorting barrier." While Dijkstra runs in $O(m+n \log n)$, the new method operates in $O(m \log^{2/3} n)$, offering faster performance on large, sparse graphs. Link: <https://arxiv.org/pdf/2504.17033>

Working Principle

1. Assign distance value 0 to the source vertex and ∞ to all other vertices.
2. Mark all vertices as unvisited.
3. Select the unvisited vertex having the smallest tentative distance value.
4. Mark the selected vertex as visited because its shortest distance from the source has been finalized.
5. For every adjacent vertex of the selected vertex, calculate the new distance using:

$$\text{New Distance} = \text{dist}[u] + w(u, v)$$

If the newly calculated distance is smaller than the currently stored distance, update the distance value.

6. Repeat the process until all vertices are visited or the smallest distance among unvisited vertices becomes infinity.

4.4.2 Algorithm

Algorithm 4.4.1

DIJKSTRA_SHORTEST_PATH(G, s, t)

Input: Graph G , source vertex s , target vertex t

Output: Shortest path distances and parent pointers

1. $dist[s] \leftarrow 0$
2. $\pi[s] \leftarrow \text{NIL}$
3. For each vertex $v \in V$:
 - (a) If $v \neq s$:
 - i. $dist[v] \leftarrow \infty$
 - ii. $\pi[v] \leftarrow \text{undefined}$
 - (b) **ENQUEUE**(v, Q) (insert v into queue Q)
4. While Q is not empty:
 - (a) $u \leftarrow$ vertex in Q having minimum $dist[u]$
 - (b) If $u = t$ then break
 - (c) **DEQUEUE**(u, Q) (remove u from queue Q)
 - (d) For each adjacent node v of u :
 - i. $val \leftarrow dist[u] + \text{weight}(u, v)$
 - ii. If $val < dist[v]$:
 - A. $dist[v] \leftarrow val$
 - B. $\pi[v] \leftarrow u$
5. **Return** $dist, \pi$

Note:-

For each edge (u, v) with weight $w(u, v)$:

if $dist[u] + w(u, v) < dist[v]$ **then** $dist[v] \leftarrow dist[u] + w(u, v)$

4.4.3 Time Complexity of Dijkstra's Algorithm

Let V = number of vertices and E = number of edges.

First loop (initialization)

There are $|V|$ vertices. Each operation inside the loop (setting distance, parent, enqueue) is $O(1)$. Hence total time = $O(|V|)$.

Second loop (while loop)

1. **Extract min:** In the array implementation, we scan all remaining vertices in Q to find the minimum. In the worst case, Q has $|V|$ vertices $\rightarrow O(|V|)$. This happens $|V|$ times $\rightarrow O(|V|^2)$.
2. **Inner for loop (update neighbours):** A vertex can have at most $|V| - 1$ neighbours in a fully connected graph. So the inner loop can run up to $O(|V|)$ per vertex, which is included in the same $O(|V|^2)$ bound.

Hence,

$$O(|V|) + O(|V|^2) + O(|V|^2)$$

$$T_{\text{array}}(|V|, |E|) = O(|V|^2).$$

Complexity with Different Data Structures

Data Structure	Total Time
Array / Linear search	$O(V^2)$
Binary Heap	$O((V + E) \log V)$
Fibonacci Heap	$O(E + V \log V)$

Note:-

Dijkstra's algorithm cannot handle negative edge weights.

4.4.4 Space Complexity of Dijkstra's Algorithm

The space complexity of this algorithm is $O(V)$, where V is the number of vertices in the graph.

The space is consumed by the following:

- **dist[] array** – stores one distance value per vertex $\Rightarrow O(V)$.
- **$\pi[]$ array** – stores one parent pointer per vertex $\Rightarrow O(V)$.
- **Queue Q** – the initialization loop enqueues every vertex, so it holds at most $O(V)$ elements at any time $\Rightarrow O(V)$.

Adding these together:

$$O(V) + O(V) + O(V) = O(V).$$

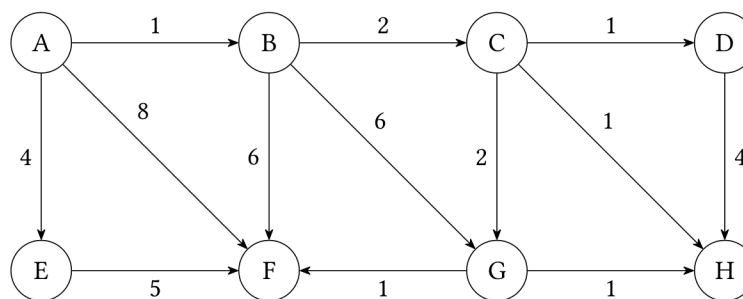
The choice of priority queue implementation (binary heap, Fibonacci heap, or unsorted array) affects only the *time* complexity – the queue never holds more than V vertices, so the space complexity remains $O(V)$ in all cases.

Note:-

Including the input graph. The $O(V)$ figure is the *auxiliary* (extra) space used by the algorithm and does not count the storage for the graph G itself. If the graph storage is included, the total space is: $O(V + E)$

Problem:

Suppose Dijkstra's algorithm is run on the following graph, starting at node A.



Show the final shortest path tree with a table showing the intermediate distance values of all the nodes at each iteration of the algorithm.

Solution:

Here, source vertex is A.

dist[u] indicates distance of vertex u from source.

$\pi[u]$ indicates parent / previous node of u .

Initialization:

$$\text{dist}[\text{source}] = 0 \Rightarrow \text{dist}[A] = 0$$

$$\pi[\text{source}] = \text{undefined} \Rightarrow \pi[A] = \text{NIL}$$

$$\text{dist}[B] = \text{dist}[C] = \text{dist}[D] = \text{dist}[E] = \text{dist}[F] = \text{dist}[G] = \text{dist}[H] = \infty$$

$$\pi[B] = \pi[C] = \pi[D] = \pi[E] = \pi[F] = \pi[G] = \pi[H] = \text{NIL}$$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	∞	∞	∞	∞	∞	∞	∞
$\pi[u]$	NIL	NIL	NIL	NIL	NIL	NIL	NIL	NIL

Iteration 1:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = A$

$\text{Adjacent}[A] = \{B, E, F\}$

$$\text{val}[B] = \text{dist}[A] + \text{weight}(A, B) = 0 + 1 = 1$$

Here, $\text{val}[B] < \text{dist}[B]$, so update $\text{dist}[B]$

$\text{dist}[B] = 1$, and $\pi[B] = A$

$$\text{val}[E] = \text{dist}[A] + \text{weight}(A, E) = 0 + 4 = 4$$

Here, $\text{val}[E] < \text{dist}[E]$, so update $\text{dist}[E]$

$\text{dist}[E] = 4$ and $\pi[E] = A$

$$\text{val}[F] = \text{dist}[A] + \text{weight}(A, F) = 0 + 8 = 8$$

Here, $\text{val}[F] < \text{dist}[F]$, so update $\text{dist}[F]$

$\text{dist}[F] = 8$ and $\pi[F] = A$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	∞	∞	4	8	∞	∞
$\pi[u]$	NIL	A	NIL	NIL	A	A	NIL	NIL

Iteration 2:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = B$

$\text{Adjacent}[B] = \{C, F, G\}$

$$\text{val}[C] = \text{dist}[B] + \text{weight}(B, C) = 1 + 2 = 3$$

Here, $\text{val}[C] < \text{dist}[C]$, so update $\text{dist}[C]$

$\text{dist}[C] = 3$ and $\pi[C] = B$

$$\text{val}[F] = \text{dist}[B] + \text{weight}(B, F) = 1 + 6 = 7$$

Here, $\text{val}[F] < \text{dist}[F]$, so update $\text{dist}[F]$

$\text{dist}[F] = 7$ and $\pi[F] = B$

$$\text{val}[G] = \text{dist}[B] + \text{weight}(B, G) = 1 + 6 = 7$$

Here, $\text{val}[G] < \text{dist}[G]$, so update $\text{dist}[G]$

$\text{dist}[G] = 7$ and $\pi[G] = B$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	∞	4	7	7	∞
$\pi[u]$	NIL	A	B	NIL	A	B	B	NIL

Iteration 3:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = C$
 $\text{Adjacent}[C] = \{D, G\}$

$\text{val}[D] = \text{dist}[C] + \text{weight}(C, D) = 3 + 1 = 4$

Here, $\text{val}[D] < \text{dist}[D]$, so update $\text{dist}[D]$

$\text{dist}[D] = 4$ and $\pi[D] = C$

$\text{val}[G] = \text{dist}[C] + \text{weight}(C, G) = 3 + 2 = 5$

Here, $\text{val}[G] < \text{dist}[G]$, so update $\text{dist}[G]$

$\text{dist}[G] = 5$ and $\pi[G] = C$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	4	4	7	5	∞
$\pi[u]$	NIL	A	B	C	A	B	C	NIL

Iteration 4:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = E$

$\text{Adjacent}[E] = \{F\}$

$\text{val}[F] = \text{dist}[E] + \text{weight}(E, F) = 4 + 5 = 9$

Here, $\text{val}[F] > \text{dist}[F]$, so no change in table.

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	4	4	7	5	∞
$\pi[u]$	NIL	A	B	C	A	B	C	NIL

Iteration 5:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = D$

$\text{Adjacent}[D] = \{G, H\}$

$\text{val}[G] = \text{dist}[D] + \text{weight}(D, G) = 4 + 1 = 5$

Here, $\text{val}[G] = \text{dist}[G]$, so don't update $\text{dist}[G]$

$\text{val}[H] = \text{dist}[D] + \text{weight}(D, H) = 4 + 4 = 8$

Here, $\text{val}[H] < \text{dist}[H]$, so update $\text{dist}[H]$

$\text{dist}[H] = 8$ and $\pi[H] = D$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	4	4	7	5	8
$\pi[u]$	NIL	A	B	C	A	B	D	D

Iteration 6:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = G$

$\text{Adjacent}[G] = \{F, H\}$

$\text{val}[F] = \text{dist}[G] + \text{weight}(G, F) = 5 + 1 = 6$

Here, $\text{val}[F] < \text{dist}[F]$, so update $\text{dist}[F]$

$\text{dist}[F] = 6$ and $\pi[F] = G$

$\text{val}[H] = \text{dist}[G] + \text{weight}(G, H) = 5 + 1 = 6$

Here, $\text{val}[H] < \text{dist}[H]$, so update $\text{dist}[H]$

$\text{dist}[H] = 6$ and $\pi[H] = G$

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	4	4	6	5	6
$\pi[u]$	NIL	A	B	C	A	G	C	G

Iteration 7:

u = unprocessed vertex in Q having minimum $\text{dist}[u] = F$

$\text{Adjacent}[F] = \{ \}$

So, no change in table.

Iteration 8:

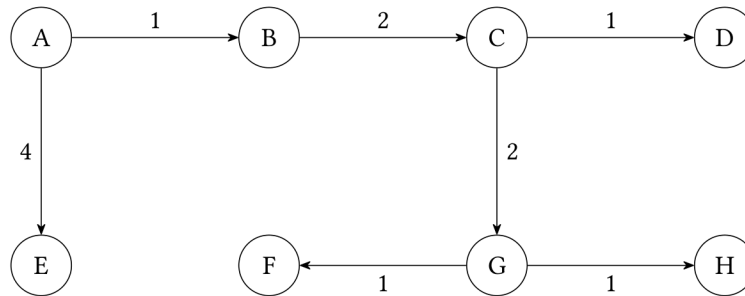
u = unprocessed vertex in Q having minimum $\text{dist}[u] = H$

$\text{Adjacent}[H] = \{ \}$

So, no change in table.

Vertex u	A	B	C	D	E	F	G	H
$\text{dist}[u]$	0	1	3	4	4	6	5	6
$p[u]$	NIL	A	B	C	A	G	C	G

We can easily derive the shortest path tree for the given graph from the above table. In the table, $p[u]$ indicates the parent node of vertex u . The shortest path tree is shown in the following figure.

**Assignment:**

Suppose Dijkstra's algorithm is run on the following graph, starting at node A. Find the final shortest path tree.

